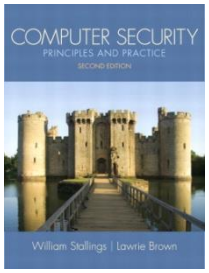
# אבטחת סייבר

(אבטחת מידע/מחשבים/מערכות ממוחשבות/...)




המצגת הינה לשימוש פנימי בלבד

אין לצלם או לעשות שימוש שלא במסגרת הקורס

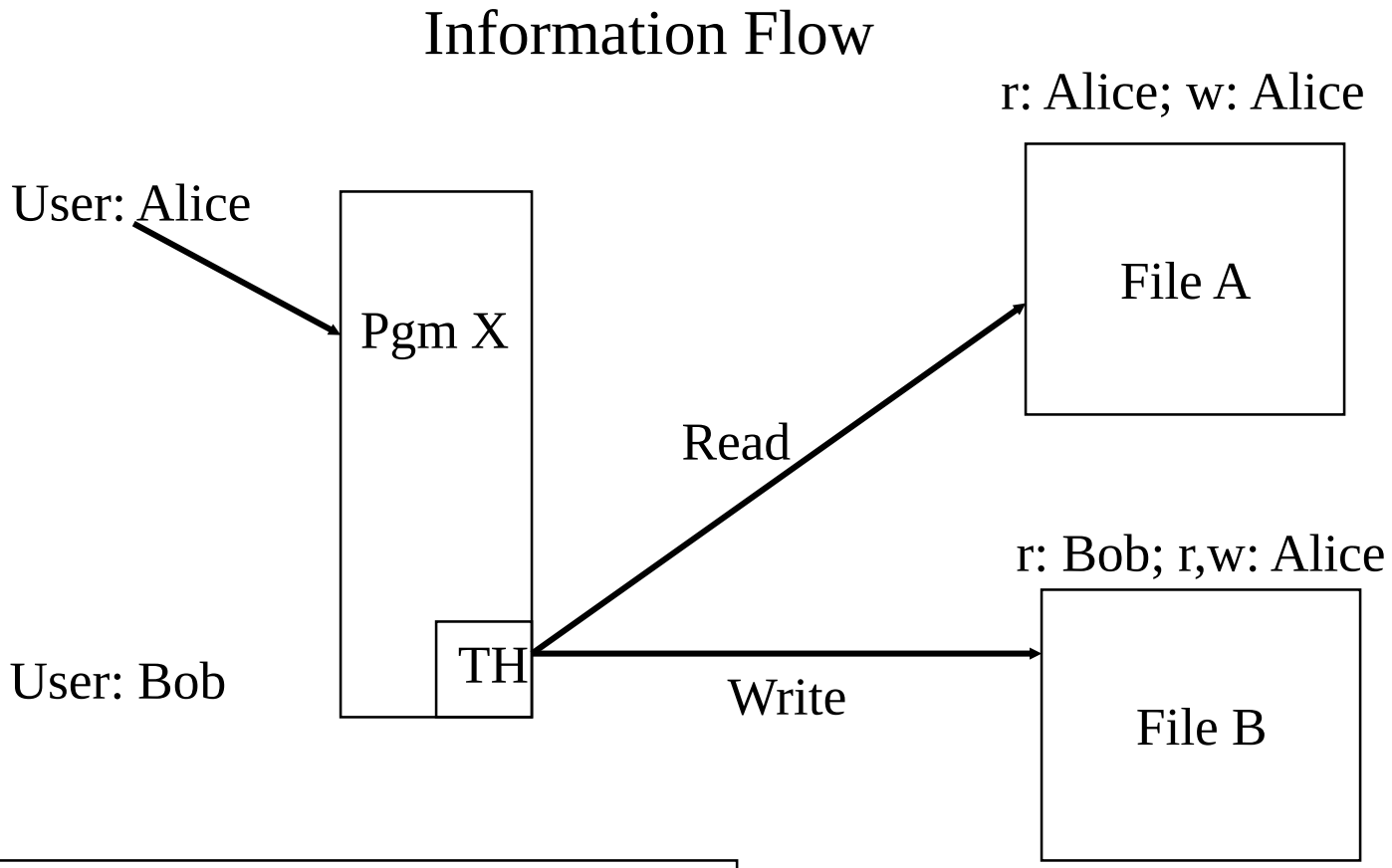
# בפעם הקודמת

- SSL
- KERBEROS
- בקרת גישה (Access Control)
  - DAC
    - subjects, object, access rights
    - Access Matrix
    - ACL, C-List
  - RBAC
    - RBAC0-RBAC3
  - MAC
    - BLP

# היום

- BLP – חסרונות
- ערוצים נסתרים
- דיון: בקרת גישה (Access Control) ב-Linux ו-Windows
- הקשחת מערכות הפעלה
- BO ופגיעויות נוספות

# The Information Flow Problem of DAC (II)




User Bob can read contents of the file A copied to file B

# Bell-LaPadula (BLP) Confidentiality Model (1973)

By David Bell and Leonard LaPadula

Hierarchical

Addresses the confidentiality of information in a system

Mandatory access control

Formalize the U.S. Department of Defense (DoD) multilevel security (MLS) policy; the basis for the Orange Book

Simple security property – read capability only

A subject can read information at or below his level of secrecy, but cannot read any information above his level of secrecy (no read up)

Star property – write capability only

A subject can write information at or above his level of secrecy, but cannot read any information above his level of secrecy (no write down)

Higher secrecy level

Assigned level

Lower secrecy level

# Problems of BLP

1. The need for a Trusted subject (to write non-secret info.)

**Solution**: **high water mark**! – get the sensitivity level of the highest level accessed object during the program execution so far…

2. The problem of covert channels.
3. No access-control when all subjects and all objects are at the same level.

# Covert Channels

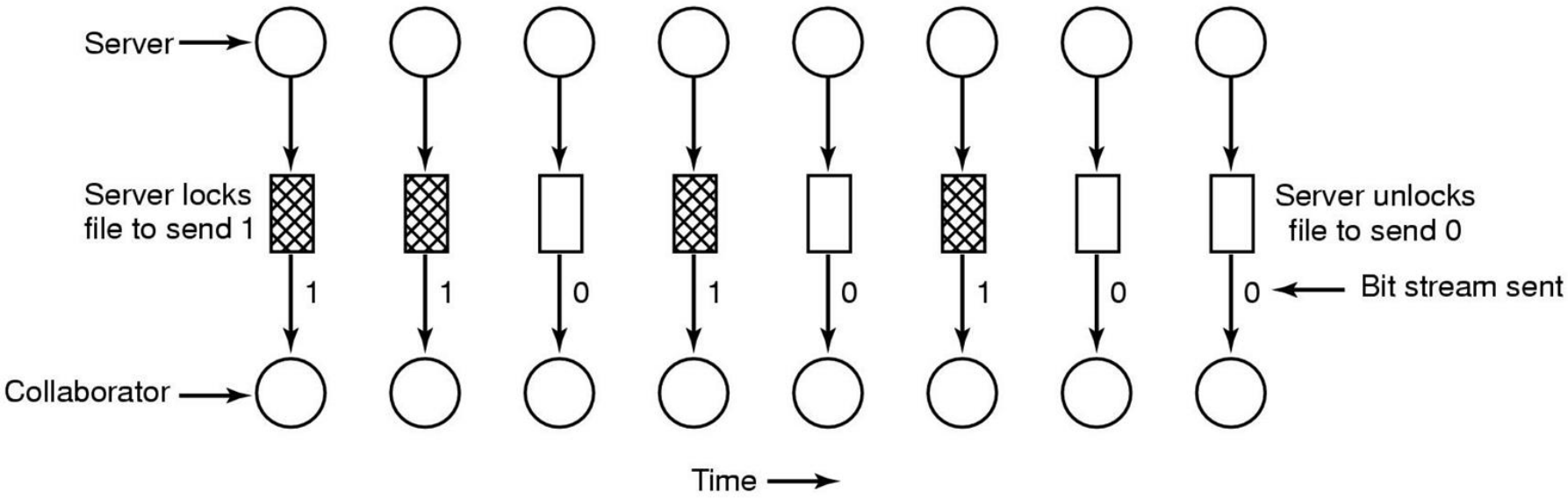
## Control of Information Flow

U1:

```
if profit < 0
then delete file T
else begin
        write file T, 'text';
        close file T;
end
```

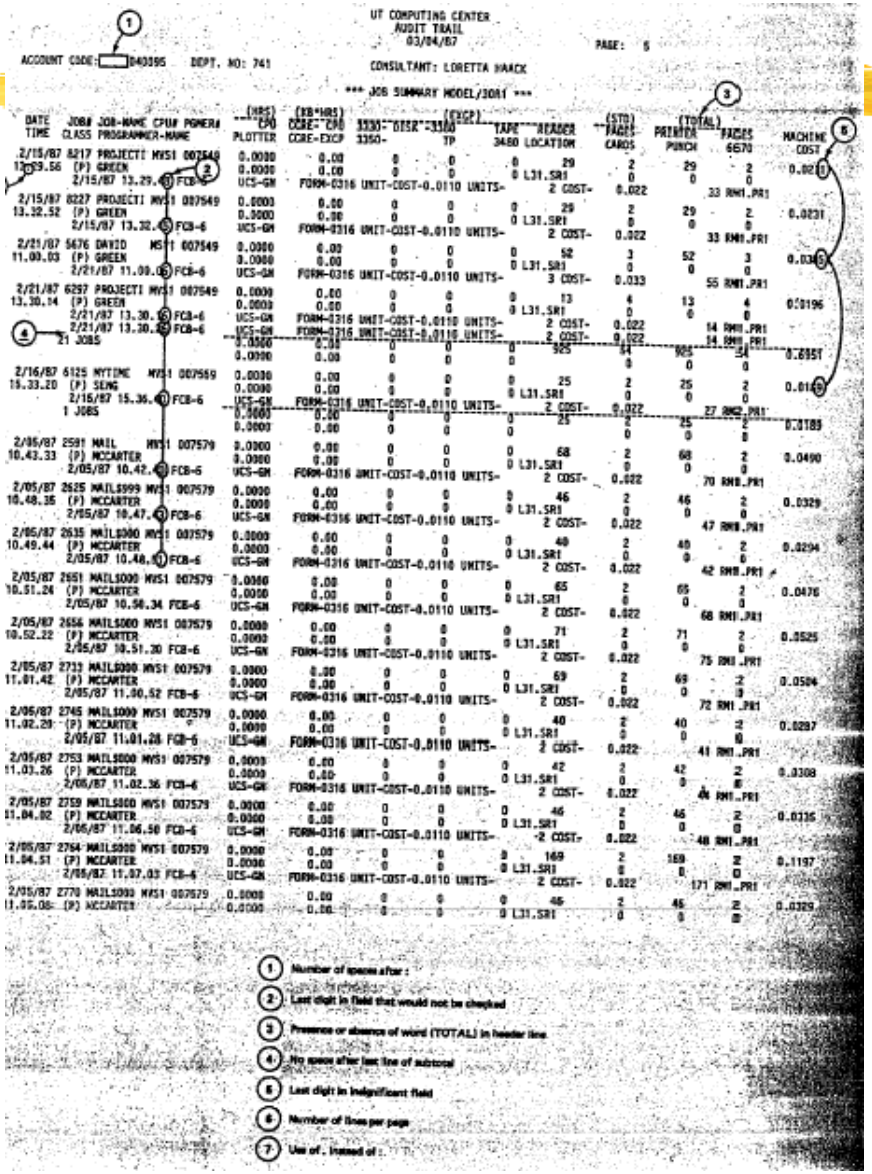
The existence of file T (regardless of its label) gives information about the value of *profit*. Its content even more …

# Covert Channels




A covert channel using file locking

# Covert Channels



UT COMPUTING CENTER
AUDIT TRAIL
03/04/87

PAGE: 5

ACCOUNT CODE: [redacted]040395 DEPT. NO: 741

CONSULTANT: LORETTA HAACK

*** JOB SUMMARY MODEL/3081 ***

| DATE TIME | JOB# JOB-NAME CLASS | CPU# PROGRAMER-NAME | (HRS) CPU PLOTTER | (KB*HRS) CORE-CPU CORE-EXCP | 3330- | 3350- | (EXCP) TP | TAPE | READER 3480 LOCATION | (STD) PAGES CARDS | PRINTER PUNCH | (TOTAL) PAGES 6670 | MACHINE COST |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2/15/87 13.29.56 | 8217 | PROJECT1 MVS1 007549 (P) GREEN | 0.0000 | 0.00 | 0 | 0 | 0 | 29 | | 2 | 29 | 2 | 0.0231 |
| | | 2/15/87 13.29.4[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 33 RM1.PR1 | |
| 2/15/87 13.32.52 | 8227 | PROJECT1 MVS1 007549 (P) GREEN | 0.0000 | 0.00 | 0 | 0 | 0 | 29 | | 2 | 29 | 2 | 0.0231 |
| | | 2/15/87 13.32.4[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 33 RM1.PR1 | |
| 2/21/87 11.00.03 | 5676 | DAVID MVS1 007549 (P) GREEN | 0.0000 | 0.00 | 0 | 0 | 0 | 52 | | 3 | 52 | 3 | 0.0[illegible] |
| | | 2/21/87 11.00.0[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 3 COST- | | 0.033 | | 55 RM1.PR1 | |
| 2/21/87 13.30.14 | 6297 | PROJECT1 MVS1 007549 (P) GREEN | 0.0000 | 0.00 | 0 | 0 | 0 | 13 | | 4 | 13 | 4 | 0.0196 |
| | | 2/21/87 13.30.1[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | 2/21/87 13.30.2[illegible] FCB-6 | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 14 RM1.PR1 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 14 RM1.PR1 | |
| | 21 | JOBS | 0.0000 | 0.00 | 0 | 0 | 0 | 905 | | 64 | 905 | 64 | 0.6951 |
| | | | 0.0000 | 0.00 | 0 | 0 | 0 | | | 0 | 0 | 0 | |
| 2/16/87 15.33.20 | 6125 | MYTIME MVS1 007569 (P) SENG | 0.0000 | 0.00 | 0 | 0 | 0 | 25 | | 2 | 25 | 2 | 0.0[illegible] |
| | | 2/16/87 15.36.4[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 27 RM2.PR1 | |
| | | | 0.0000 | 0.00 | 0 | 0 | 0 | 25 | | 2 | 25 | 2 | 0.0189 |
| | | | 0.0000 | 0.00 | 0 | 0 | 0 | | | 0 | 0 | 0 | |
| 2/05/87 10.43.33 | 2591 | MAIL MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 68 | | 2 | 68 | 2 | 0.0490 |
| | | 2/05/87 10.42.4[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 70 RM8.PR1 | |
| 2/05/87 10.48.35 | 2625 | MAILS999 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 46 | | 2 | 46 | 2 | 0.0329 |
| | | 2/05/87 10.47.4[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 47 RM8.PR1 | |
| 2/05/87 10.49.44 | 2635 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 40 | | 2 | 40 | 2 | 0.0294 |
| | | 2/05/87 10.48.5[illegible] FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 42 RM8.PR1 | |
| 2/05/87 10.51.24 | 2651 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 65 | | 2 | 65 | 2 | 0.0476 |
| | | 2/05/87 10.50.34 FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 68 RM1.PR1 | |
| 2/05/87 10.52.22 | 2656 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 71 | | 2 | 71 | 2 | 0.0525 |
| | | 2/05/87 10.51.20 FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 75 RM1.PR1 | |
| 2/05/87 11.01.42 | 2713 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 69 | | 2 | 69 | 2 | 0.0504 |
| | | 2/05/87 11.00.52 FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 72 RM1.PR1 | |
| 2/05/87 11.02.20 | 2745 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 40 | | 2 | 40 | 2 | 0.0287 |
| | | 2/05/87 11.01.28 FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 41 RM1.PR1 | |
| 2/05/87 11.03.26 | 2753 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 42 | | 2 | 42 | 2 | 0.0308 |
| | | 2/05/87 11.02.36 FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 44 RM1.PR1 | |
| 2/05/87 11.04.02 | 2759 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 46 | | 2 | 46 | 2 | 0.0335 |
| | | 2/05/87 11.06.50 FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | -2 COST- | | 0.022 | | 48 RM1.PR1 | |
| 2/05/87 11.04.51 | 2764 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 169 | | 2 | 169 | 2 | 0.1197 |
| | | 2/05/87 11.07.03 FCB-6 | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |
| | | | UCS-GM | FORM-0316 UNIT-COST-0.0110 UNITS- | | | | 2 COST- | | 0.022 | | 171 RM1.PR1 | |
| 2/05/87 11.05.06 | 2770 | MAIL1000 MVS1 007579 (P) MCCARTER | 0.0000 | 0.00 | 0 | 0 | 0 | 46 | | 2 | 46 | 2 | 0.0329 |
| | | | 0.0000 | 0.00 | 0 | 0 | 0 | 0 | L31.SR1 | 0 | 0 | 0 | |

1. Number of spaces after :
2. Last digit in field that would not be checked
3. Presence or absence of word (TOTAL) in header line
4. No space after last line of subtotal
5. Last digit in insignificant field
6. Number of lines per page
7. Use of . instead of :

# Covert Channels

- Pictures appear the same
- Picture on right has text of 5 Shakespeare plays
  - encrypted, inserted into low order bits of color values

Zebras

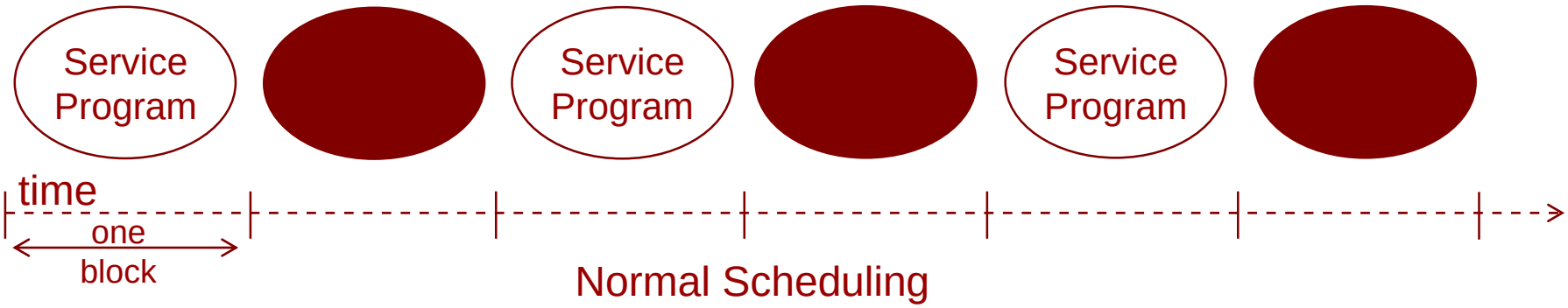
Hamlet, Macbeth, Julius Caesar
Merchant of Venice, King Lear

# Covert Channels – CPU usage

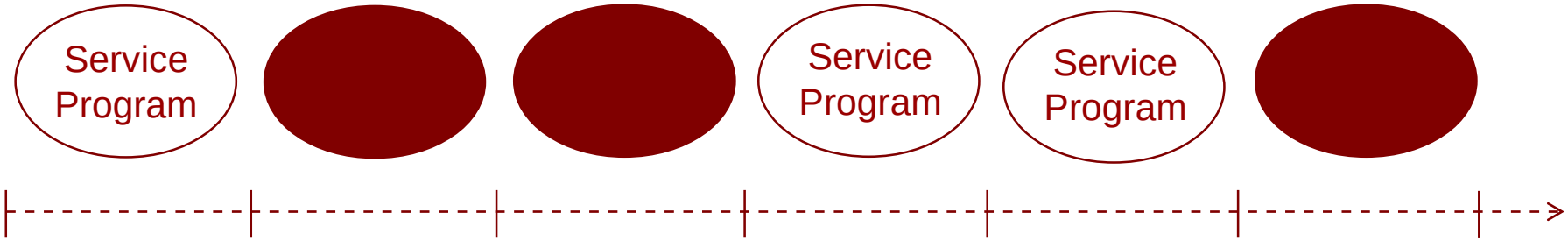
- proc copy(x: int 0..1 class { x };
  var y: int 0..1 class Low);

```
begin
    y := 0;
    while x = 0 do (* nothing *);
    y := 1;
end
```

# Covert Timing Channel




Normal Scheduling




Service Program Communicating 101

# Fighting Covert Channels (after Detection)

- Deletion of the channel if feasible
- Add random noise on the channel
- Audit the channel
- Ignore, danger is not very serious

# The HRU Model

- Six primitive operations for manipulating subjects, objects, and the access matrix:
  - enter $\underline{r}$ into $M_{so}$
  - delete $\underline{r}$ from $M_{so}$
  - create subject $s$
  - delete subject $s$
  - create object $o$
  - delete object $o$

- Commands in HRU model (examples):

**command** create_file($s,f$)
    create $f$
    enter $\underline{o}$ into $M_{s,f}$
    enter $\underline{r}$ into $M_{s,f}$
    enter $\underline{w}$ into $M_{s,f}$
end

**command** grant_read($s,p,f$)
    if $\underline{o}$ in $M_{s,f}$
    then enter $\underline{r}$ in $M_{p,f}$
end

# HRU Security

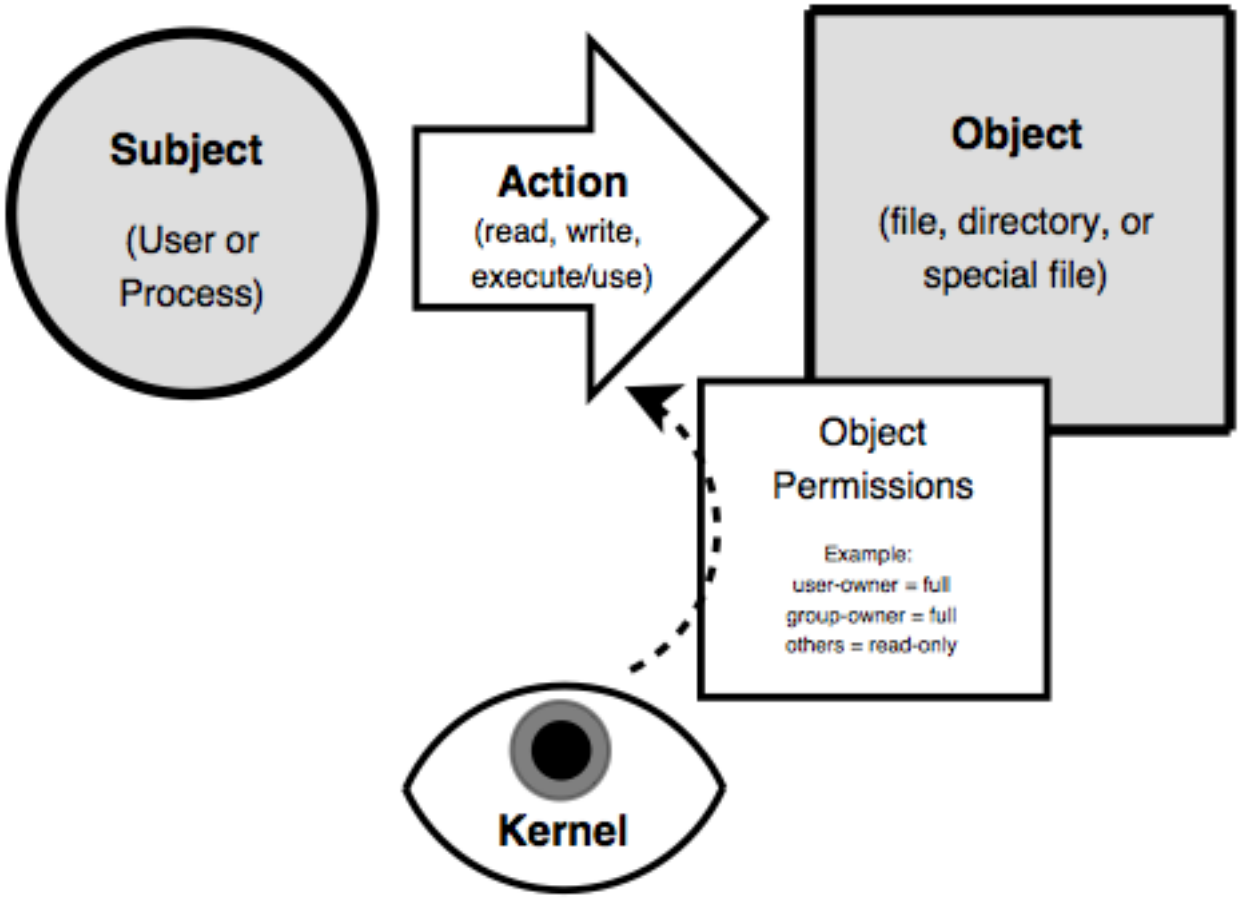
- Is there an algorithm to decide whether a given set of commands can lead to an unsafe state?
- The general problem is **Undecideable!**
  - Can be reduced to the halting problem.
- For restricted models, the chances of success are better.
  - **Theorem**. Given a mono-operational authorisation system, an access matrix *M*, and a right *r*, verifying the safety of *M* with respect to *r* is decidable.

# HRU – Summary

- Do not memorize the individual undecidability theorems for the HRU variants.
- Remember the important message: The more expressive the security model, the more difficult it is to verify security.
- You don't have to prove theorems based on computational complexity theory for this course but it helps to appreciate the challenges in formally verifying security.

# דיון: בקרת גישה (Access Control) ב-Linux ו-Windows

# Linux Security Transactions



# UID/GID

- User identifiers (UIDs) and group identifiers (GIDs) are the principals.

- A UID (GID) is a 16-bit number; examples:

  0: root
  1: bin
  2: daemon
  8: mail
  9: news
  261: diego

- UID values differ from system to system

- Superuser (root) UID is always zero.

# Users and Groups

- user's details are kept in `/etc/password`

  `maestro:x:200:100:Maestro Edward Hizzersands:/home/maestro:/bin/bash`

- additional group details in `/etc/group`

  `conductors:x:100:`

  `pianists:x:102:maestro,volodya`

- use **useradd**, **usermod**, **userdel** to alter

* x is a placeholder for maestro's password (which is actually stored in /etc/shadow)

# File Permissions

- files have two owners: a user & a group
- each with its own set of permissions
- with a third set of permissions for other
- permissions are to read/write/execute in order user/group/other, cf.

```
ls -l
-rw-rw-r--  1  maestro user 35414 Mar
 25 01:38 baton.txt
```

- set using chmod command

```
   chmod u=rwx, g=rx, o=r myfile
or:
   chmod 754 myfile
```

# Directory Permissions

- read = list contents
- write = create or delete files in directory
- execute = use anything in or change working directory to this directory
- e.g.

```
$ chmod g+rx extreme_casseroles
$ ls -l extreme_casseroles
 drwxr-x--- 8 biff  drummers 288
 Mar 25 01:38 extreme_casseroles
```

# UNIX File Access Control

- "set user ID"(SetUID) or "set group ID"(SetGID)
  - system temporarily uses rights of the file owner / group in addition to the real user's rights when making access control decisions
  - enables privileged programs to access files / resources not generally accessible
- sticky bit
  - on directory limits rename/move/delete to owner
- superuser
  - is exempt from usual access control restrictions

# Sticky Bit

- originally used to lock file in memory
- now used on directories to limit delete
  - if set must own file or dir to delete
  - other users cannot delete even if have write
- set using chmod command with +t flag, e.g.

```
chmod +t extreme_casseroles
```

- directory listing includes t or T flag

```
drwxrwx--T  8  biff  drummers  288  Mar
  25 01:38 extreme_casseroles
```

- only apply to specific directory not child dirs

# SetUID and SetGID

- setuid bit means program "runs as" owner
  - no matter who executes it
- setgid bit means run as a member of the group which owns it
  - again regardless of who executes it
- "run as" = "run with same privileges as"
- *are very dangerous* if set on file owned by root or other privileged account or group
  - only used on executable files, not shell scripts
- SUDO: ---s--x—x … /usr/bin/sudo
  - **/etc/sudoers**

# setuid root Vulnerabilities

- a **setuid root** program runs as root
  - *no matter who executes it*
- used to provide unprivileged users with access to privileged resources
- must be very carefully programmed
- if can be exploited due to a software bug
  - may allow otherwise-unprivileged users to use it to wield unauthorized root privileges
- distributions now minimize setuid-root programs
- system attackers still scan for them!

# Kernel vs User Space

- Kernel space
  - refers to memory used by the Linux kernel and its loadable modules (e.g., device drivers)
- User space
  - refers to memory used by all other processes
- since kernel enforces Linux DAC and security critical to isolate kernel from user
  - so kernel space never swapped to disk
  - only root may load and unload kernel modules

# Unix File System Security – Violating Security Principles

- Principle of **Least Privilage** (group access)
- Principle of **Safe Defaults** (default accounts & passwords)
- Principle of **Need to Know** (Others access, Super-user power)
- Principle of **Accountability** (setUid)

Always there is **Tradeoff**:

**Security / Convenience / Performance!**

# Linux System Hardening

- consider how to mitigate Linux security risks at system and application levels
- first look at OS-level security tools and techniques that protect the entire system

# OS Installation

- security begins with O/S installation
- especially what software is run
  - since unused applications liable to be left in default, un-hardened and un-patched state
- generally should not run:
  - X Window system, RPC services, R-services, inetd, SMTP daemons, telnet etc
- also have some initial system s/w configuration:
  - setting root password
  - creating a non-root user account
  - setting an overall system security level (usually initial file-permission settings)
  - enabling a simple host-based firewall policy
  - Using extended ACLs
  - enabling SELinux (Mandatory access control)

# Windows Access Control

- Access control in Windows applies to objects: files, registry keys, Active Directory objects,…
- More complex than access control in a file system.
- Means for structuring policies in complex systems: groups, roles, inheritance; access rights beyond read and write.
- Identify principals, subjects, and objects.
- Access rules: where to find them, how they are evaluated.

# Access Control Lists

- two forms of access control list (ACL):
- Discretionary ACL (DACL)
  - grants or denies access to protected resources such as files, shared memory, named pipes etc
- System ACL (SACL)
  - used for auditing and to enforce mandatory integrity policy

# Access Control Lists

- objects needing protection are assigned a DACL (and possible SACL) that includes
  - SID of the object owner
  - list of access control entries (ACEs)
- each ACE includes a SID & access mask
- access mask could include ability to:
  - read, write, create, delete, modify, etc
- access masks are object-type specific
  - e.g. service abilities are create, enumerate

# Security Descriptor (SD)

- data structure with object owner, DACL, & SACL
  - e.g.

```
Owner: CORP\Blake
ACE[0]: Allow CORP\Paige Full Control
ACE[1]: Allow Administrators Full Control
ACE[2]: Allow CORP\Cheryl Read, Write and Delete
```

- have no implied access, if there is no ACE for requesting user, then access is denied
- applications must request correct type of access
  - if just request "all access" when need less (e.g. read) some user's who should have access will be denied

# More SD's & Access Checks

- each ACE in the DACL determines access
- an ACE can be an allow or a deny ACE
- Windows evaluates each ACE in the ACL until access is granted or explicitly denied
- so deny ACEs come before allow ACEs
  - default if set using GUI
  - explicitly order if create programmatically
- when user attempts to access a protected object, the O/S performs an access check
  - comparing user/group info with ACE's in ACL

# Question

- What is the difference between Empty DACL and No DACL?
- Answer:
- Empty DACL: nobody is granted any permission
- No DACL (NULL DACL): everybody gets all access

# Windows System Hardening

- process of shoring up defenses, reducing exposed functionality, disabling features
  - known as attack surface reduction
  - use 80/20 rule on features
  - not always achievable
  - e.g. requiring RPC authentication in XP SP2
  - e.g. strip mobile code support on servers
- servers easier to harden:
  1. are used for very specific and controlled purposes
  2. perceive server users are administrators with better computer configuration skills than typical users

# Windows Vulnerabilities

- Windows, like all O/S's, has security bugs
  - and bugs have been exploited by attackers to compromise customer operating systems
- Microsoft now uses process improvement called the Security Development Lifecycle
  - net effect approx 50% reduction in bugs
- Windows Vista used SDL start to finish
- IIS v6 (in Windows Server 2003) had only 3 vulnerabilities in 4 years, none critical

# Windows Security Defenses

- attackers are now criminals rather than young, anarchic miscreants, and are highly motivated by money
- have categories of security defenses:
  - account defenses
  - network defenses
  - browser defenses
  - buffer overrun defenses

# Account Defenses

- user accounts can have privileged SIDs
- least privilege dictates that users operate with just enough privilege for tasks
- Windows XP users in local Administrators
  - for application compatibility reasons
  - can use "Secondary Logon" to run apps (run as…)
  - also restricted tokens reduce per-thread privilege
- Windows Vista reverses default with UAC
  - users prompted to perform a privileged operation
  - unless admin on Server

# Network Defenses

- need more than user defenses
- vulnerable to attack via network service
- have IPSec and IPv6 with authenticated network packets enabled by default in Windows Vista
  - IPv4 also enabled by default, expect less use
- have built-in software firewall
  - block inbound connections on specific ports
    - Vista can allow local net access only
  - optionally block outbound connections (Vista)
  - default was off (XP) but now default on (Vista)

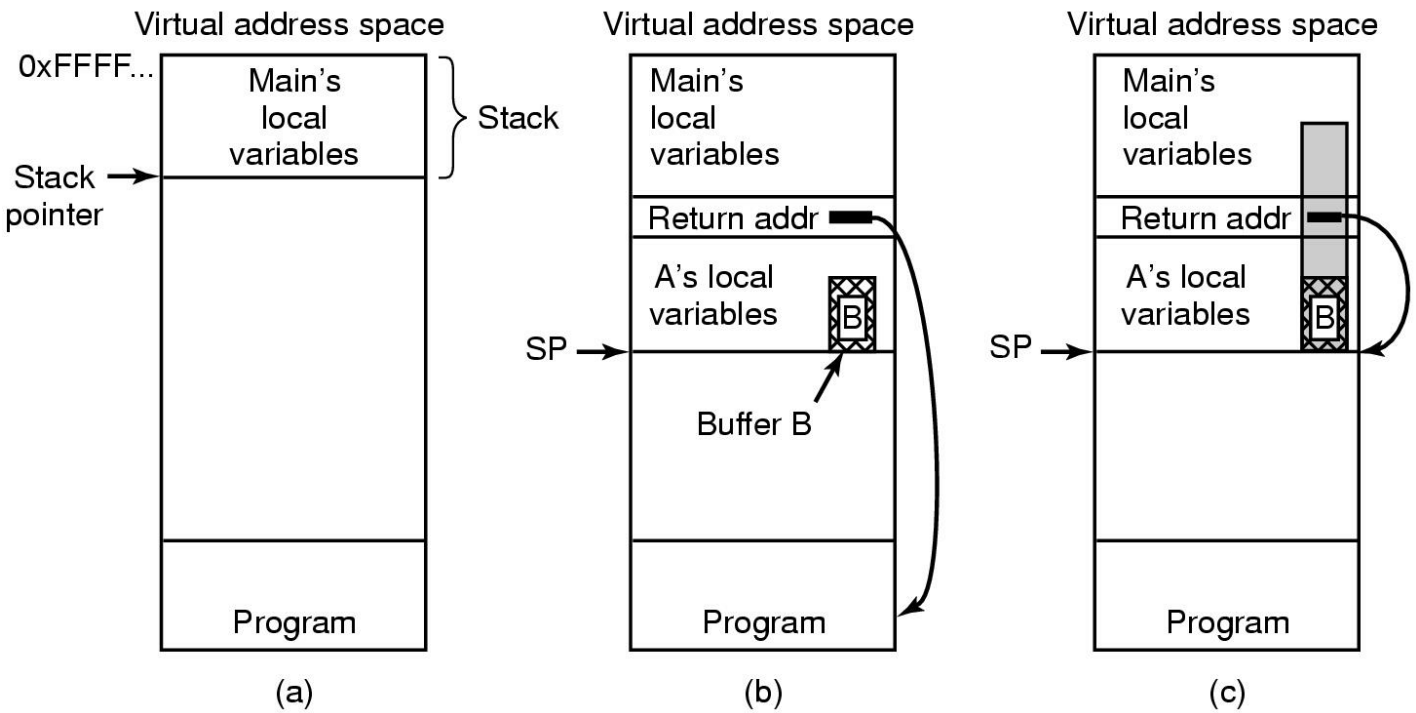
# Browser Defenses

- web browser is a key point of attack
  - via script code, graphics, helper objects
- Microsoft added many defenses to IE7
  - ActiveX opt-in
    - unloads ActiveX controls by default
    - when any then first run prompts user to confirm
  - protected mode
    - IE runs at low integrity level (see earlier)
    - so more difficult for malware to manipulate O/S

# Buffer Overrun Defenses

- Many compromises exploit buffer overruns
- Many Microsoft product were exploited via BO
- Now – built in protection

# Vulnerabilities – Buffer Overflow

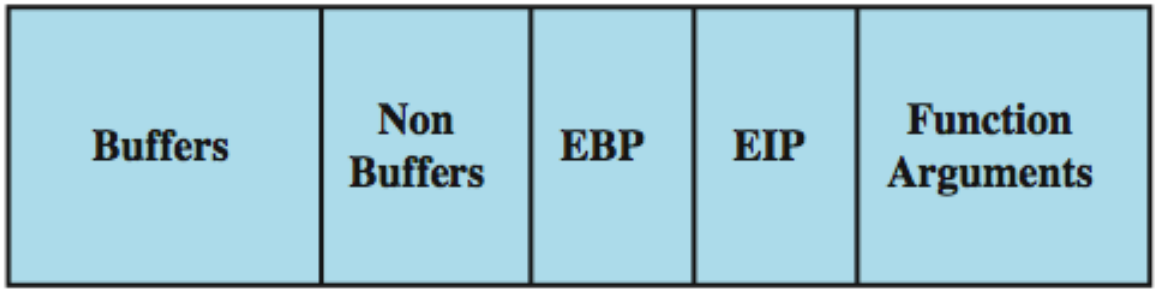



- (a) Situation when main program is running
- (b) After program *A* called
- (c) Buffer overflow shown in gray

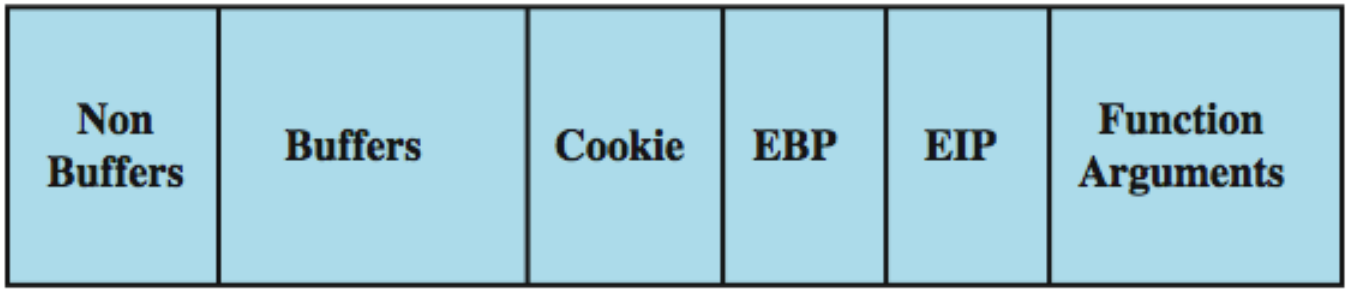
# Buffer Overrun Defenses

- many compromises exploit buffer overruns
- Windows Vista has "Stack-Based Buffer Overrun Detection (/GS)" default enabled
  - source code compiled with special /GS option
  - does not affect every function; only those with at least 4-bytes of contiguous stack data and that takes a pointer or buffer as an argument
- defends against "classic stack smash"

# Windows Stack and /GS flag




(a) Without /GS option




(b) With /GS option

# Buffer Overrun Defenses

- No eXecuteNamed (NX) / Data Execution Prevention (DEP) / eXecution Disable (XD)
  - prevent code executing in data segments
  - as commonly used by buffer overrun exploits
  - applications linked with /NXCOMPAT option
- Stack Randomization (Vista only)
  - randomizes thread stack base addresses
- Heap-based buffer overrun defenses:
  - add and check random value on each heap block
  - heap integrity checking
  - heap randomization (Vista only)

# Other Defenses

- Image Randomization
  - O/S boots in one of 256 configurations
  - makes O/S less predictable for attackers
- Service Restart Policy
  - services can be configured to restart if fail
  - great for reliability but lousy for security
  - Vista sets some critical services so can only restart twice, then manual restart needed
  - gives attacker only two attempts